EXP NO: **LEX PROGRAM TO CREATE SYMBOL TABLE MANAGER**

DATE:

Aim:

To create a lex program to create and print the symbol table manager.

Algorithm:

Step 1: Start the program.

Step 2: Define the token handling functions such as create_token, append_token, print_heading and print_and_free_list.

Step 3: Define the patterns and actions in lex.

Step 4: Skip non matching characters such as \t, \n, whitespaces etc.

Step 5: Compile and run the program using lex and gcc.

Step 6: Display the symbol table manager as output.

Step 7: Stop the program.

Source code:

```
%{
   typedef struct token {
      char name[20];
      char type[15];
      void *address;
      struct token *next;
   } token;

   token *head = NULL;
   int is_first_print = 1;

   token *create_token(const char s[20], const char t[15], void *addr) {
      token *new_token = (token *)malloc(sizeof(token));
      if (new_token == NULL) {
```

```
        fprintf(stderr, "Memory allocation failed\n");
        exit(1);
    }
    strcpy(new_token->name, s);
    strncpy(new_token->type, t, 14);
    new_token->type[14] = '\0';
    new_token->address = addr;
    new_token->next = NULL;
    return new_token;
}

void append_token(const char s[20], const char t[15], void *addr) {
    if (head == NULL) {
        head = create_token(s, t, addr);
    } else {
        token *temp = head;
        while (temp->next) {
            temp = temp->next;
        }
        temp->next = create_token(s, t, addr);
    }
}

void print_heading() {
    if (is_first_print) {
        printf("\n%-10s%-20s%-20s\n", "Type", "Variable", "Address");
        printf("-----------------------------------------------\n");
        is_first_print = 0;
    }
}
```

```
void print_and_free_list() {
    print_heading();
    token *temp = head;
    while (temp) {
        printf("%-10s%-20s%-20p\n", temp->type, temp->name, temp->address);
        token *to_free = temp;
        temp = temp->next;
        free(to_free);
    }
    head = NULL;
}

const char *keywords[] = {
    "auto", "break", "case", "char", "const", "continue", "default", "do", "double",
    "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register",
    "return", "short", "signed", "sizeof", "static", "struct", "switch", "typedef",
    "union", "unsigned", "void", "volatile", "while"
};

const int num_keywords = sizeof(keywords) / sizeof(keywords[0]);

int is_keyword(const char *word) {
    for (int i = 0; i < num_keywords; i++) {
        if (strcmp(word, keywords[i]) == 0) {
            return 1;
        }
    }
    return 0;
}
```

```
%}

%%

[a-zA-Z_][a-zA-Z0-9_]* {
    if (is_keyword(yytext)) {
        append_token(yytext, "keyword", NULL);
    } else {
        append_token(yytext, "variable", yytext);  // Store the address as the variable itself
    }
}

";" { print_and_free_list(); }

[ \t\n]+ { }

. { }

%%

int main(int argc, char *argv[]) {
    if (argc > 1) {
        FILE *input_file = fopen(argv[1], "r");
        if (input_file == NULL) {
            fprintf(stderr, "Error opening file\n");
            return 1;
        }
        yyin = input_file;
    } else {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
```

```
        return 1;
    }

    yylex();
    fclose(yyin);
    return 0;
}
```

Input C program

```
#include <stdio.h>
int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        printf("Factorial of %d is %d\n", num, factorial(num));
    }
    return 0;
}
```

Output:

```
Type      Variable            Address
-----------------------------------------------
variable  include             0x55f8b72db4f1
variable  stdio               0x55f8b72db4fa
variable  h                   0x55f8b72db500
keyword   int                 (nil)
variable  factorial           0x55f8b72db508
keyword   int                 (nil)
variable  n                   0x55f8b72db516
keyword   if                  (nil)
variable  n                   0x55f8b72db523
variable  n                   0x55f8b72db52d
keyword   return              (nil)
keyword   else                (nil)
keyword   return              (nil)
variable  n                   0x55f8b72db565
variable  factorial           0x55f8b72db569
variable  n                   0x55f8b72db573
keyword   int                 (nil)
variable  main                0x55f8b72db588
```

```
keyword   int                 (nil)
variable  num                 0x55f8b72db599
variable  printf              0x55f8b72db5a7
variable  Enter               0x55f8b72db5af
variable  a                   0x55f8b72db5b5
variable  number              0x55f8b72db5b7
variable  scanf               0x55f8b72db5c7
variable  d                   0x55f8b72db5cf
variable  num                 0x55f8b72db5d4
keyword   if                  (nil)
variable  num                 0x55f8b72db5e7
variable  printf              0x55f8b72db5fa
variable  Factorial           0x55f8b72db602
variable  is                  0x55f8b72db60c
variable  not                 0x55f8b72db60f
variable  defined             0x55f8b72db613
keyword   for                 (nil)
variable  negative            0x55f8b72db61f
variable  numbers             0x55f8b72db628
variable  n                   0x55f8b72db631
keyword   else                (nil)
variable  printf              0x55f8b72db64b
variable  Factorial           0x55f8b72db653
variable  of                  0x55f8b72db65d
variable  d                   0x55f8b72db661
```

```
variable  is                0x55f8b72db663
variable  d                 0x55f8b72db667
variable  n                 0x55f8b72db669
variable  num               0x55f8b72db66d
variable  factorial         0x55f8b72db672
variable  num               0x55f8b72db67c
keyword   return            (nil)
```

Result:

The above lex code to print the symbol table manager has been created and the output has been verified successfully.